# Trabalho Prático Parte 1

Módulos `requests` e `re` - API REST de Exercício Físico
ALPCD - Ano Letivo 2026/27

## 1. Objetivos e Organização

Este trabalho prático tem como principais objetivos:

- desenvolver competências na interação com REST APIs utilizando o módulo `requests`;
- aplicar expressões regulares com o módulo `re` para processamento e limpeza de dados;
- construir uma *Command Line Interface* (CLI) funcional com a biblioteca `typer`;
- explorar dados reais de exercício físico, nutrição e performance humana.

Neste TP, que se pretende que seja resolvido de forma incremental, **aprecia-se a imaginação e criatividade** dos grupos ao incluir funcionalidades adicionais às descritas!

Deve colocar a solução final no repositório `git` do grupo até **dia 21 de Novembro**.

O programa desenvolvido será apresentado totalmente pronto e a funcionar, e defendido por **todos os elementos do grupo** em data a definir pelos docentes.

**Atenção**

A nota do trabalho será atribuída **de acordo com a capacidade dos grupos de defenderem e explicarem o que implementaram** não pelo facto de todas as alíneas estarem resolvidas ou pelos extras apresentados.

## 2. Enunciado

Neste trabalho pretende-se interagir com a API pública de exercício físico **wger** (*Workout Manager*), que disponibiliza uma base de dados abrangente de exercícios, grupos musculares, equipamento e ingredientes nutricionais.

**API**

API base utilizada neste TP:

`https://wger.de/api/v2/`

A API não requer autenticação para leitura de dados públicos. Dedique alguns minutos a explorar a documentação interativa em:
`https://wger.de/api/v2/schema/ui`

A partir daí poderá conhecer todas as rotas disponíveis, testar pedidos, verificar os tipos de dados de saída e os parâmetros aceites por cada rota.

O produto final deverá ser uma **CLI** desenvolvida em Python que permita a um utilizador pesquisar e obter dados de exercício físico da API. Para a implementação da CLI recomenda-se o uso da biblioteca `typer`[1] pela sua simplicidade e capacidade de validação de argumentos, mas outras abordagens são aceites.

Para cada alínea abaixo, crie o comando na CLI de forma a responder ao pedido.

**a)** Comando `top`: **listar os $N$ exercícios mais recentes** da base de dados. O comando deve escrever o *output* no terminal em formato JSON, incluindo nome, categoria e equipamento necessário.

**Nota**

Use o parâmetro `language=2` da API para filtrar por inglês.

Exemplo:

```
> python fitcli.py top 10
> [{"name": "Bench Press", "category": "Chest", "equipment
   ": ["Barbell"]}, ...]
```

[1] `https://typer.tiangolo.com/`

**b)** Comando `search`: **pesquisar exercícios por músculo e/ou categoria**, com um limite $N$ de resultados. O comando deve receber como argumentos o nome do músculo ou categoria, o número de resultados a mostrar, e escrever o *output* em formato JSON. Exemplo:

```
> python fitcli.py search biceps 5
> [{"name": "Barbell Curl", "category": "Arms", ...}, ...]
```

**c)** Comando `type`: **extrair o tipo de esforço** (composto/isolado/cardio/outro) de um determinado exercício a partir do seu `exercise_id`. Essa informação deverá ser inferida a partir dos campos disponíveis na resposta da API (descrição, categoria, músculos envolvidos), utilizando **expressões regulares** ou outro método que considere relevante. O comando deve receber o `exercise_id` como argumento e escrever o resultado no terminal.
Exemplo:

```
> python fitcli.py type 192
> compound
```

**d)** Comando `equipment`: **contar ocorrências de equipamento** nas descrições dos exercícios de uma determinada categoria. O comando deve receber o nome de uma categoria e escrever no terminal uma lista ordenada por ordem decrescente de ocorrências, em formato JSON.

> **Nota**
>
> Use o módulo `re` para extrair menções a equipamento (e.g. "barbell", "dumbbell", "cable", "machine", "bodyweight") nas descrições de texto dos exercícios.

Exemplo:

```
> python fitcli.py equipment chest
> [{"barbell": 12}, {"dumbbell": 8}, {"cable": 5}, ...]
```

**e)** Comando `workout`: combina as funcionalidades das alíneas anteriores e gera um **plano de treino personalizado**.
O comando deve receber como argumentos:

- uma lista de músculos ou categorias separados por vírgula (e.g. `biceps,back`);
- o tipo de esforço pretendido: `compound`, `isolated` ou `any` (sem filtro);
- o número máximo de exercícios $N$ **por músculo/categoria**.

Adicionalmente, deve ser possível exportar o plano para o formato **CSV** com um argumento opcional `-csv`.

**Dica**

Reutilize as funções já implementadas nas alíneas anteriores - esta alínea é essencialmente uma composição das restantes. Um bom design modular torna esta alínea trivial.

Exemplos:

```
> python fitcli.py workout biceps,back compound 5
> {
>   "biceps": [
>     {"name": "Barbell Curl", "type": "isolated", "
   equipment": ["Barbell"], ...},
>     ...
>   ],
>   "back": [
>     {"name": "Barbell Row", "type": "compound", "
   equipment": ["Barbell"], ...},
>     ...
>   ]
> }

> python fitcli.py workout chest,shoulders isolated 3 --csv
    treino.csv
> Ficheiro exportado: treino.csv (6 registos)
```

**Atenção**

Durante a resolução do TP, respeite as boas práticas de utilização de APIs: adicione um cabeçalho `User-Agent` identificativo nos pedidos, verifique os status code e implemente paginação quando necessário para consumir a totalidade dos dados.